

Slide 1:

"Good morning, everyone. Today I will present the first half of my major project. I am building a system to solve hard optimization problems. Currently, I have finished the **Traveling Salesperson Problem**. In the future, I will add the **0/1 Knapsack Problem**."

Slide -2:

My project is divided into two phases. Phase 1 is about finding the shortest path between cities, which is done. Phase 2 will be about maximizing profit using the Knapsack algorithm, which is my next step. I used Java for the design and C language for speed.

Slide-3:

I started with the Traveling Salesperson Problem. It looks easy, but it is actually very hard. If you have just 12 cities, there are almost 500 million possible paths. If you try to check them all one by one, the computer will crash.

Slide-4:

To solve this, I first used the **Branch and Bound** algorithm. This method is perfect. It guarantees to find the absolute shortest path. It works by 'pruning' or cutting off bad paths early, so we don't waste time checking them.

Slide-5:

Let's look at my first algorithm, Branch and Bound. The biggest advantage here is **Accuracy**. It guarantees the absolute best solution, which is crucial for high-stakes problems like drilling circuit boards. However, the downside is **Scalability**. Because it checks so many possibilities, it consumes huge amounts of memory. For 20 cities, it would take years to solve.

Slide-6:

However, Branch and Bound is slow if we have 50 cities. So, I added a second algorithm called **Ant Colony Optimization**. This mimics how real ants find food using scent trails. It doesn't guarantee a perfect answer, but it is extremely fast for large maps.

Slide -7:

On the other hand, we have Ant Colony Optimization. Its main strength is **Speed and Scalability**. I can run this on a map with 100 cities, and it solves it in seconds. The trade-off is that it is **Probabilistic**. It doesn't guarantee the mathematical minimum, but for real-world logistics like Uber or Amazon, a solution that is 99% optimal in 1 second is better than a 100% optimal solution that takes 100 years

Slide-8:

This is the most important part of my work. I compared both algorithms. For small maps, Branch and Bound is the winner because it is exact. But for large maps, Ant Colony is the winner because it doesn't crash. My system lets the user choose the best tool for the job

Slide-9:

As I mentioned, this is an ongoing project. My next immediate goal is to implement the **0/1 Knapsack Problem**. I will use **Dynamic Programming** to solve it. I will add this feature to the same C backend I have already built

Slide-:

To conclude, I have successfully built a system that solves the TSP using two different methods. I have set up a strong foundation with Java and C, and I am ready to start the Knapsack implementation next. Thank you for listening